

Tema 4.1 — Composición

1) Idea clave: “A tiene-un / tiene-varios B”

La composición es una forma de construir entidades más grandes a partir de otras más pequeñas: una cosa contiene a otra (“tiene-un” o “tiene-varios”).

Se usa para modelar estructuras donde un objeto incluye a otros como parte de su “estado”.

2) Composición en C (struct anidadas)

En C, se expresa de forma natural con struct dentro de struct. Un ejemplo típico es una Línea que está formada por dos Puntos, y funciones para operar: distancia entre puntos y longitud de línea.

La composición aquí es estructural: la struct Línea “contiene” dos struct Punto.

3) Composición en Java: clases + encapsulación + (posible) inmutabilidad

En Java se modela igual, pero con clases y encapsulación: Línea tiene dos Punto.

Diferencia importante: en Java puedes reforzar la ocultación de información usando private, y puedes crear objetos inmutables usando atributos final y evitando setters. Así, un Punto no cambia sus coordenadas y una Línea no cambia sus extremos una vez creada.

La idea es que cada clase tenga su responsabilidad: Punto calcula distancias y Línea calcula su longitud delegando en Punto.

4) Multiplicidad (cuántos con cuántos)

La multiplicidad indica cuántas instancias de una clase se relacionan con una instancia de otra (por ejemplo, 1, 0..1, 1..*, etc.).

En el ejemplo Línea–Punto:

De Línea → Punto: multiplicidad 2 (una línea está compuesta exactamente por dos puntos).

De Punto → Línea: multiplicidad 0..* (un punto puede no pertenecer a ninguna línea o pertenecer a muchas).

5) Composición fuerte vs composición débil (agregación/asociación)

Hay dos “intensidades” de relación “tiene-un”:

a) Composición fuerte (composición “propia”)

El objeto contenido no tiene sentido sin el contenedor.

Su ciclo de vida está ligado: si “muere” el contenedor, “mueren” sus partes (conceptualmente).

b) Composición débil = agregación/asociación

El objeto contenido puede existir independientemente.

Puede ser compartido por varios contenedores.

El ciclo de vida no está ligado.

Por convención: composición suele referirse a la fuerte, y agregación/asociación a la débil.

6) Dependencia (no es composición)

Si una clase “usa” otra solo puntualmente (parámetro, retorno, variable local o new dentro de un método), eso se llama dependencia, no composición.

La diferencia clave es que en la dependencia no queda una relación estructural persistente como parte del estado del objeto (no “la tiene guardada” como atributo).

7) Implementar fuerte vs débil (cómo se refleja en el código)

En el ejemplo Línea–Punto:

Fuerte: la Línea crea/controla sus Punto (por ejemplo, el constructor recibe coordenadas primitivas y construye puntos dentro). Así los puntos “viven” dentro de la línea.

Débil: los Punto se crean fuera y la Línea solo guarda referencias a ellos (pueden compartirse).

8) ¿Por qué en Java no “se destruye” explícitamente?

En Java no verás que el contenedor destruya a sus objetos: existe el recolector de basura (garbage collector), que libera memoria cuando los objetos dejan de ser accesibles.

En composición fuerte, si la Línea deja de tener referencias y los Punto no están referenciados por nadie más, todos se vuelven candidatos a recolección.

9) Ejemplo conceptual de agregación con invariantes (Departamento–Profesor–Director)

Aquí se modela una composición débil: un Departamento tiene varios Profesor y además tiene un director que debe cumplir una invariante: siempre hay director y el director siempre pertenece a la lista de profesores.

Se sugiere proteger invariantes con excepciones si se intentan operaciones inválidas (por ejemplo, eliminar al director sin cambiarlo).

También se insiste en no romper encapsulación: no exponer directamente el array/lista interna, sino métodos como “cuántos hay” y “obtener por posición”.

10) Usar List en vez de arrays: qué te ahorras y cómo preservar encapsulación

Al cambiar arrays por List, se evita mucha gestión manual: contador, desplazamientos al borrar, tamaño fijo, etc.; List aporta add, remove, get, size.

Pero la invariante (director $\in$ profesores) sigue siendo tu responsabilidad.

Si existiera un método para devolver “todos los profesores”, devolver la lista interna rompe encapsulación, porque permitiría modificarla desde fuera y saltarse reglas.

Soluciones mencionadas: devolver una vista no modificable (`Collections.unmodifiableList`) o una copia.

11) Composición recursiva

Una composición recursiva ocurre cuando una clase contiene otra instancia de sí misma (ej.: `Persona` tiene una madre que es otra `Persona`).

Para que sea manejable, se define un caso base (por ejemplo `madre == null`) y se recomienda inmutabilidad (final, sin setters) para coherencia.

Ejemplos clásicos citados: listas enlazadas, árboles, directorios y expresiones.

12) Composición/relación bidireccional

Una relación bidireccional significa que ambas clases se referencian (`Departamento` → `Profesor` y `Profesor` → `Departamento`).

No basta con “añadir el atributo inverso”: hay que mantener consistencia en ambas direcciones al añadir/quitar profesores (asignar o poner a null el departamento del profesor, etc.).